

T.C.
IZMIR UNIVERSITY OF ECONOMICS



FACULTY OF ENGINEERING

The Integrated Assignment Environment Project
Design REPORT

Talat Karasakal - 20220602048

Uğur Emin Baynal - 20220602408

Sıla Karabağ - 20230602039

Gözde Yılıkyılmaz -20230602077

Sezen Çetinkaya - 20220602021

Advisor: Asst. Prof. Dr. İlker KORKMAZ

IZMIR, May 2026

Table of Contents

1. Executive Summary.....	3
2. Introduction.....	3
2.1 Purpose of the Project.....	3
2.2 Problem Statement.....	3
2.3 Objectives.....	4
2.4 Scope.....	4
3. Project Overview.....	4
3.1 Background.....	4
3.2 Target Users.....	5
3.3 Key Features.....	5
3.4 Assumptions and Constraints.....	5
4. Requirements Analysis.....	6
4.1 Requirements.....	6
4.2 Non-Functional Requirements.....	7
5. System Design.....	7
5.1 Overall Architecture.....	7
5.2 Module Breakdown.....	8
5.3 Data Flow Description.....	10
5.4 Database Design.....	11
5.5 UI/UX Design Principles.....	11
6. UML Diagrams.....	14
7. Technology Stack.....	15
8. Implementation Plan.....	15
8.1 Module Assignments.....	15
8.2 Development Phases.....	16
9. Testing Strategy.....	16
9.1 Unit Testing.....	16
9.2 Integration Testing.....	17
9.3 System Testing.....	17
9.4 User Acceptance Testing (UAT).....	18
10. Future Improvements.....	20
11. Conclusion.....	20

1. Executive Summary

The Integrated Assignment Environment (IAE) is a standalone desktop application developed for the CE316 course project to automate the evaluation of student programming assignments on Windows. It eliminates the time-consuming and error-prone process of manually compiling, running, and comparing student submissions by providing a fully automated evaluation pipeline, from extracting ZIP files to generating Pass/Fail results.

The system is built on a flexible configuration mechanism that supports multiple programming languages such as C, Java, and Python. Lecturers can define compilation and execution steps, and reuse or share these configurations across different projects through import/export functionality. Each project combines a configuration, student submissions, and expected outputs into a reusable evaluation unit.

The application is implemented in Java with a JavaFX GUI and uses SQLite for local data storage, requiring no external server. It is distributed as a self-contained Windows installer with all dependencies bundled, including a built-in help manual.

The design follows the MVC architecture to ensure modularity, maintainability, and testability. All project requirements are addressed, and the system is planned to be delivered as a fully functional, installable application.

2. Introduction

2.1 Purpose of the Project

This document presents the software design of the Integrated Assignment Environment (IAE), a desktop application developed for the CE316 course project. The IAE helps lecturers manage and evaluate programming assignments by automating extraction, compilation, execution, and output comparison of student submissions. This reduces manual grading effort and improves consistency.

2.2 Problem Statement

Manual evaluation of programming assignments is time-consuming and error-prone. Lecturers must individually extract submissions, compile or interpret code, run tests, compare outputs, and record results. This repetitive workflow can lead to inconsistencies and unfair grading. Additionally, there is no lightweight, offline tool tailored to this academic process that supports multiple languages and generates persistent reports. The IAE is designed to address this gap.

2.3 Objectives

The project aims to:

- Develop a Windows-based GUI application to automate assignment evaluation.
- Support multiple programming languages via configurable environments.
- Automate the full pipeline: extraction, compilation/interpreting, execution, and output comparison for batch submissions.
- Generate and store per-student evaluation reports (errors, outputs, pass/fail results).
- Ensure robustness by continuing execution even if individual submissions fail.
- Provide a self-contained Windows installer without external dependencies.

2.4 Scope

The IAE supports both compiled (e.g., C, Java) and interpreted (e.g., Python) languages via configurable setups. It processes ZIP-based student submissions in batch, executes automated evaluation, and presents results through a JavaFX GUI. Data is stored locally using SQLite, and the system is distributed as a standalone Windows installer. A built-in help manual is included in the application.

3. Project Overview

3.1 Background

Programming assignments are a core part of computer science education, providing students with practical experience and lecturers with a way to assess learning outcomes. However, evaluation is typically performed manually and lacks dedicated tooling.

In a standard workflow, lecturers collect student submissions as ZIP files, extract them individually, compile or run the code, provide inputs, compare outputs with expected results, and record grades. For medium-sized classes, this process becomes time-consuming and inconsistent, as fatigue and repetition can affect evaluation quality.

The CE316 project identifies this issue and introduces the Integrated Assignment Environment (IAE) as a targeted solution. The IAE focuses solely on the evaluation phase, runs locally without external services, supports multiple languages via configuration files, and generates persistent evaluation reports. It complements existing submission platforms rather than replacing them.

3.2 Target Users

The primary users are course lecturers who design assignments, define evaluation criteria, and review results. They are expected to have basic technical knowledge (e.g., compiler usage, command-line arguments) but not to write scripts or manage infrastructure. The IAE provides a GUI-based workflow that hides system-level complexity.

Teaching assistants form a secondary user group. They typically execute prepared projects and configurations rather than creating new ones. The system supports this through simple project open/save functionality without requiring additional role management.

3.3 Key Features

The IAE includes the following core features aligned with project requirements:

- **Project management:** Create, open, and manage evaluation projects linked to language configurations.
- **Language configuration management:** Define compilation/execution settings for different languages; import/export configurations for reuse.
- **Automated batch processing:** Detect and extract student ZIP submissions from a directory and process them automatically.
- **Evaluation pipeline:** Compile/interpret code, execute with predefined inputs, and compare outputs against expected results.
- **Result storage and reporting:** Store compilation results, runtime outputs, comparison results, and errors per student for later review.
- **Built-in documentation:** Provide an offline help manual accessible from the application's Help menu.

3.4 Assumptions and Constraints

Assumptions	Constraints
Target computers run Windows 10/11 with Java installed.	The software must not depend on any external server (e.g., MySQL, PostgreSQL).
Compilers and interpreters (e.g., gcc, javac, python) are installed and accessible via the system PATH on the lecturer's machine.	The installer must be fully self-contained, bundling all required libraries (e.g., JavaFX, SQLite JDBC driver).

Students correctly zip their submissions such that extracting the archive produces a directory named after their student ID, containing the expected source file.	The zip and unzip system utilities are NOT assumed to be present; the application must handle ZIP extraction internally via a bundled Java library.
The lecturer provides a valid expected output file that matches the exact format produced by a correct student program.	The application must function entirely offline with no internet dependency at runtime.
Only one lecturer operates the application on a given machine at a time; no multi-user concurrency is required.	The source filename expected within each student's submission directory must be defined in the configuration and cannot vary between students within the same project.

4. Requirements Analysis

4.1 Requirements

ID	Requirement	Type
R1	Windows installer that deploys all required files and adds a desktop shortcut.	System
R2	Help menu item opens a user manual.	System
R3	Users can create a project that uses an existing or new configuration.	User
R4	Users can create, edit and remove configurations.	User
R5	Users can import and export configurations as files.	User
R6	Software processes ZIP files for each student, in batch, from a directory.	User
R7	Software compiles or interprets source code using the project configuration.	User

R8	Software compares student program output against an expected output.	User
R9	Software displays per-student results.	User
R10	Users can save and reopen projects to view their results at any time.	User

4.2 Non-Functional Requirements

ID	Non-Functional Requirement	Priority	Source
NR-01	The system shall continue processing remaining students if an error occurs for one submission (robustness).	High	R6, R7
NR-02	The system shall complete batch evaluation of 30 submissions in a reasonable time frame.	Medium	R6
NR-03	Results must be displayed in a clear, readable layout accessible to non-technical users.	Medium	R9
NR-04	The application must function fully offline with no internet dependency at runtime.	High	R1
NR-05	All project data, results, and error logs must persist reliably across sessions.	High	R10

5. System Design

5.1 Overall Architecture

The IAE follows the Model-View-Controller (MVC) architectural pattern, layered over a service and data-access tier. This pattern was chosen because it enforces a clean separation between the domain data, the application logic, and the user interface — making each layer independently testable and allowing the GUI to be modified without touching the evaluation pipeline, and vice versa.

The four layers are structured as follows:

Model Layer. The domain model consists of Project, Configuration, StudentResult, TestCase, and Report. These are plain Java objects with no dependency on JavaFX or the database. They represent the core data structures of the application and are shared across all other layers via the interfaces defined in the Core module.

View Layer. All user-facing screens are built with JavaFX and defined as FXML files. The view has no business logic; it only renders data passed to it by the Controller layer and forwards user events (button clicks, menu selections) back to the appropriate controller.

Controller Layer. JavaFX controller classes (MainController, ConfigurationController, ResultsController) sit between the view and the service layer. They handle user events, validate input, invoke the appropriate service or DAO, and update the view with the result. Controllers are the only layer that holds a reference to both the GUI and the backend.

Service and Data Layer. The SubmissionProcessor, CommandRunner, ZipExtractor, and OutputComparator form the service layer — they contain the core evaluation logic and have no knowledge of the GUI. Below them, the DAO classes (ProjectDAO, ConfigurationDAO, StudentResultDAO) and DatabaseHelper provide all persistence through SQLite via JDBC.

This layered structure ensures that the entire evaluation pipeline can be invoked and tested from plain JUnit tests without launching the JavaFX application, which is essential for the integration and unit testing strategy described in Section 9.

5.2 Module Breakdown

The IAE is decomposed into eight functional modules, each with a clearly defined responsibility. The boundaries between modules are enforced through shared interfaces defined in the Core module, ensuring that each module can be developed and tested independently before integration.

Module	Responsibility	Key Classes / Files
Core	Defines the shared domain model and interfaces that all other modules depend on. Provides the Logger, Validator, and base Runner abstraction. Must be completed before parallel development begins.	Project.java, Configuration.java, StudentResult.java, TestCase.java, Report.java, Logger.java, Validator.java
GUI / View Layer	All JavaFX screens, FXML layout files, CSS stylesheets, and Controller classes. Handles all user interaction including the project creation wizard, configuration	MainController.java, ConfigurationController.java, ResultsController.java, *.fxml, styles.css

	editor, results view, and menu actions.	
Database Layer	Provides SQLite-backed persistent storage for all project and result data. Manages the JDBC connection lifecycle and initialises the schema on first run.	DatabaseHelper.java, ProjectDAO.java, ConfigurationDAO.java, StudentResultDAO.java
Configuration Manager	Handles creation, editing, deletion, import, and export of language configuration profiles. Configurations are serialised to portable JSON files for sharing and backup.	ConfigurationDAO.java, ConfigurationController.java
Submission Processor	Orchestrates the full per-student evaluation pipeline: coordinates ZipExtractor, CommandRunner, and OutputComparator in sequence for each student ZIP, and passes results to the database.	SubmissionProcessor.java
Execution Engine	Invokes external system commands (compilers, interpreters, compiled executables) via ProcessBuilder, captures stdout and stderr, enforces configurable timeouts, and returns a structured ProcessResult.	CommandRunner.java, ProcessResult.java
Files & Comparator	Handles all file system operations: ZIP extraction via a bundled Java library (no system unzip assumed), workspace directory management per student, source file location, and line-by-line output comparison with whitespace normalisation.	ZipExtractor.java, OutputComparator.java

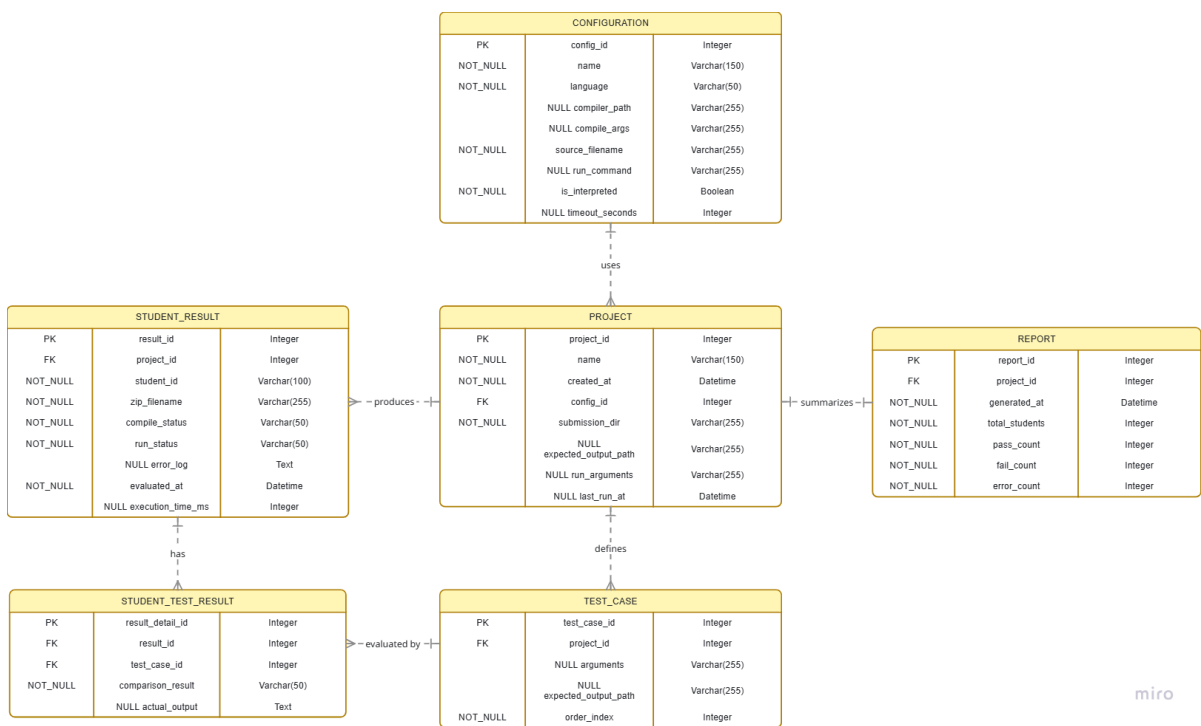
Report Generator	Aggregates per-student results into a summary report, computes pass/fail/error counts, generates an HTML report file, and provides the data model consumed by the Results view.	Report.java, ResultsController.java
------------------	---	-------------------------------------

The dependency flow between modules is strictly one-directional: the GUI layer depends on Controllers, Controllers depend on Service modules (SubmissionProcessor, ConfigurationManager), Service modules depend on the DAO layer and utility modules (CommandRunner, ZipExtractor, OutputComparator), and all layers depend on the Core domain model. No lower layer references a higher one, which keeps the architecture clean and independently testable.

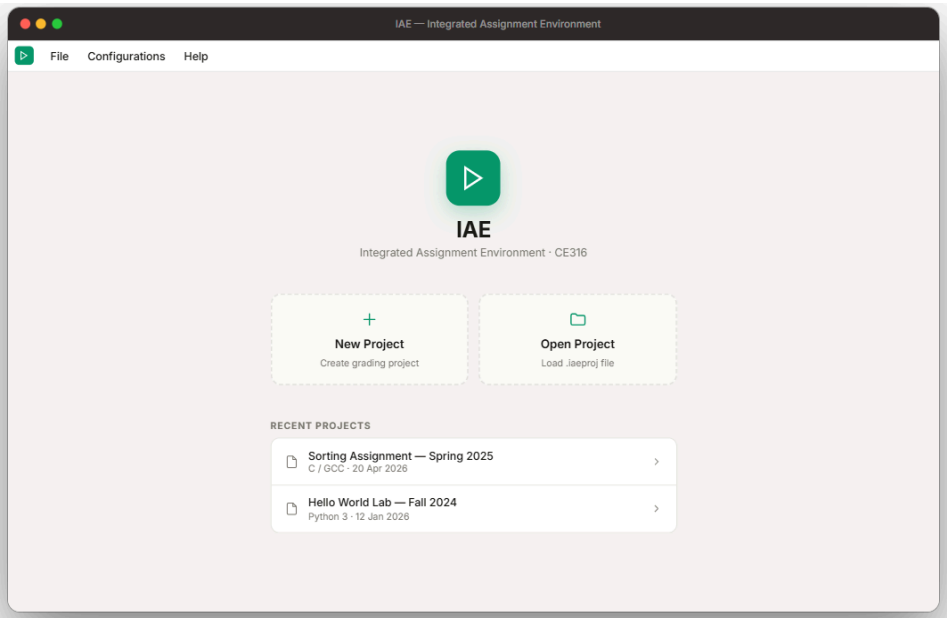
5.3 Data Flow Description

When the lecturer clicks Run, the MainController passes the project and submission directory to the SubmissionProcessor. For each ZIP file found, the ZipExtractor unpacks it into a temporary workspace named after the student ID. The CommandRunner then invokes the compiler defined in the project's Configuration; if compilation fails, the error is recorded and processing moves to the next student. On success, the compiled program is executed with the configured arguments and its stdout is captured. The OutputComparator compares this output against the lecturer's expected output file and returns PASS or FAIL. The result — including status, output, and any error log — is persisted to SQLite via StudentResultDAO and the results view is updated. If any step throws an exception, it is caught, logged, and the pipeline continues with the next submission.

5.4 Database Design



5.5 UI/UX Design Principles



IAE — Integrated Assignment Environment — Ce_316_Lab008

FileConfigurationsHelp

Ce_316_Lab008

C / GCC - /home/lecturer/cs316/submissions/

1 / 6

ALLPASSFAILCOMPILEMISSINGTIMEOUT

STUDENT ID	NAME	STATUS	DURATION	ERROR
STU20210001	Alice Yildiz	PASS	0.82s	-

New Project

1

2

3

4

DetailsConfigurationFiles & ArgsReview

Project Name

CE_316_Lab008

Comparator

Exact Match

CancelNext

New Project

✓

2

3

4

DetailsConfigurationFiles & ArgsReview

Select Configuration

>_C / GCC

GCC compiler, C99 standard

compile: gcc main.c -o main

run: ./main

>_Python 3

CPython interpreter

run: python3 main.py

>_Java 17

OpenJDK 17 LTS

compile: javac Main.java

run: java Main

BackNext

New Project

✓

✓

3

4

DetailsConfigurationFiles & ArgsReview

Submissions Directory

/home/lecturer/cs316/submissions/

Browse

Expected Output File

/home/lecturer/cs316/expected-output.txt

Browse

Command-line Arguments (one per line)

apple
banana
cherry
mango

← Back

Next →

New Project

✓

✓

✓

4

DetailsConfigurationFiles & ArgsReview

Project Name

CE_316_Lab008

Configuration

C / GCC

Comparator

exact

Submissions

/home/lecturer/cs316/submissions/

Expected Output

/home/lecturer/cs316/expected-output.txt

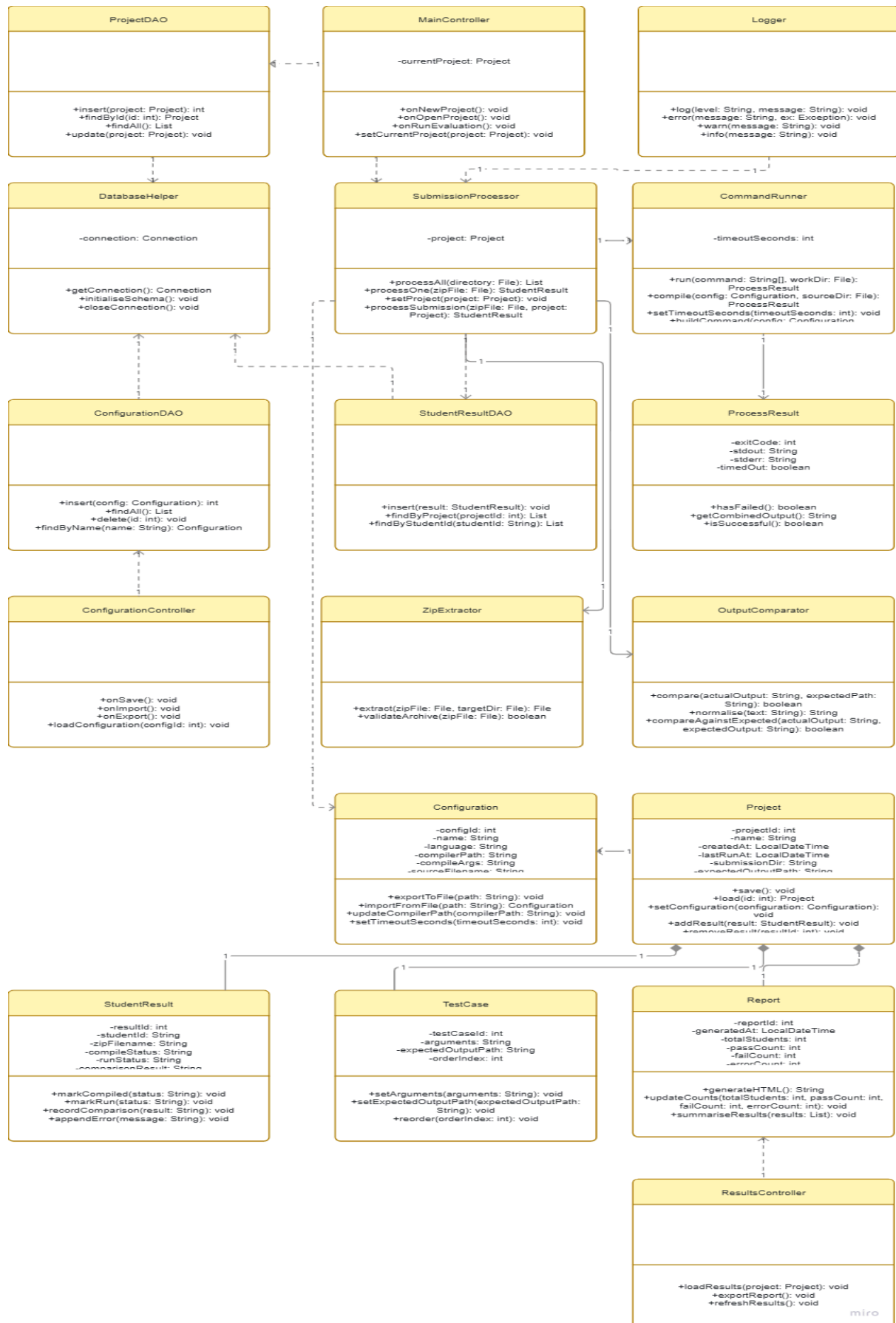
📄

Project will be saved as CE_316_Lab008.iaeproj

← Back

Create Project

6. UML Diagrams



7. Technology Stack

Category	Technology	Version
Programming Language	Java	21 LTS
GUI Framework	JavaFX	21
Database	SQLite via JDBC	3.45.3.0
Build Tool	Gradle	8.7
Installer	Inno Setup	6.3.3
Version Control	Github	github.com/SezenCetinkaya/CE_316_Project

8. Implementation Plan

8.1 Module Assignments

The eight functional modules described in §5.2 are grouped into five implementation packages, each owned by one team member who is fully responsible for the design, implementation, and unit testing of that package. Module 5 (Core + Installer) is owned by the team lead and serves as the shared foundation all other modules depend on; it is developed first.

#	Module	Components	Owner
1	GUI	JavaFX + FXML + Controllers + CSS	Uğur Emin Baynal
2	DB + Reports	SQLite + DAO + Import/Export + HTML Report + Stats	Sıla Karabağ
3	Execution + Plugins	ProcessRunner + Compile/Run + Plugin System	Talat Karasakal

4	Files + Comparator	ZIP + Workspace + FileLocator + Output Comparator	Sezen Çetinkaya
5	Core + Installer	Model + Interfaces + Runner + Logger + Template + Validator + Inno Setup	Gözde Yılıkyılmaz

8.2 Development Phases

Phase	Name	Key Deliverables
Phase 1	Design & Architecture	Design document, UML diagrams, DB schema, module interfaces agreed.
Phase 2	Core + Interfaces	Domain model, Logger, Validator, Runner abstraction — foundation for all other modules.
Phase 3	Parallel Module Development	GUI, DB+Reports, Execution+Plugins, Files+Comparator developed simultaneously against shared interfaces defined by Module 5.
Phase 4	Integration	All modules connected; GUI wired to backend; end-to-end pipeline runs on real student ZIPs.
Phase 5	Testing & Bug Fixes	Unit, integration, and system tests executed; all critical bugs resolved.
Phase 6	Installer & Documentation	Inno Setup installer built and tested on clean VM; Help manual finalised.

9. Testing Strategy

9.1 Unit Testing

Unit testing in the IAE focuses on verifying the correctness of individual components in isolation, independent of the GUI and the file system where possible. The primary framework used is JUnit 5, integrated into the project via Gradle.

The most critical units to test are the OutputComparator, the CommandRunner, and the ZIP extraction logic. For the OutputComparator, tests will verify that two identical strings return PASS, that strings differing in content return FAIL, and that differences caused only by

trailing whitespace or line endings are handled consistently — since student programs running on Windows may produce `\r\n` line endings rather than `\n`. For the `CommandRunner`, tests will confirm that a valid system command (e.g., `echo hello`) produces the expected stdout, that a nonexistent command is caught and reported as an error rather than crashing the application, and that a process exceeding a defined timeout is terminated gracefully. For ZIP extraction, tests will cover a well-formed archive, an archive with a missing expected source file, and a corrupt or empty archive — each should produce a clearly recorded error without propagating an unhandled exception.

Configuration serialisation will also be unit tested: a `Configuration` object written to a file and read back must produce an identical object, ensuring that import/export round-trips are lossless.

9.2 Integration Testing

Integration testing verifies that the individually correct modules work correctly when connected. The primary integration test is the full evaluation pipeline: given a prepared student ZIP file containing a known correct `main.c`, the test invokes the `SubmissionProcessor` end-to-end extraction, compilation via `CommandRunner`, execution with arguments, and output comparison via `OutputComparator` and asserts that the resulting `StudentResult` object carries a `PASS` status, a non-empty output field, and an empty error log.

A second integration test covers the database layer: after a complete pipeline run, the test queries the SQLite database directly and confirms that the `StudentResult` record was persisted with the correct student ID, project ID, status, and timestamp. This ensures that the `ReportManager` correctly commits results to storage and does not silently discard them.

A third integration test verifies the configuration import/export flow with the project system: a configuration is created programmatically, exported to a file, the application state is reset, the file is imported, and a new project is created using the imported configuration. The test asserts that the pipeline executes without errors using the re-imported configuration, confirming end-to-end fidelity.

9.3 System Testing

System testing exercises the complete, installed application as the lecturer would use it. These tests are performed manually on a clean Windows 10/11 machine with Java preinstalled (per project assumptions), using the installer produced by Inno Setup, with no development environment present.

The primary system test uses a prepared directory of five student ZIP files with the following mix: one submission that compiles and produces the correct output, one that compiles but produces incorrect output, one that contains a compilation error, one that is missing the expected source file entirely, and one that is a corrupt or invalid ZIP. After the lecturer creates

a project, selects the C configuration, points the IAE to the directory, and clicks Run, the expected outcome is five result entries in the results view — each with an accurate status (PASS, FAIL, COMPILE ERROR, FILE NOT FOUND, EXTRACTION ERROR) and no application crash or hang at any point.

Additional system tests verify: that closing and reopening the project displays the same results without re-running the pipeline; that a configuration exported on one machine can be imported on a second machine and used in a new project without modification; and that the Help menu opens the user manual correctly. Each of these tests is recorded in a checklist that maps directly to R1–R10.

9.4 User Acceptance Testing (UAT)

The UAT session follows a scripted scenario: the evaluator installs the application from the installer, launches it from the desktop shortcut, creates a language configuration for C from scratch, creates a new project using that configuration, imports a second pre-prepared configuration from a file, runs a batch evaluation over a directory of ten student ZIPs, reviews the results table, reopens the project after restarting the application, and finally opens the Help menu to locate a specific section of the manual. Each step is marked as Pass or Fail on the checklist below.

Test ID	Test Description	Precondition	Expected Result	Type
T-01	Valid C file compiles without error	ZIP contains correct main.c	Status: COMPILE SUCCESS	Unit
T-02	File with syntax error is flagged	ZIP contains broken main.c	Status: COMPILE ERROR; error log populated	Unit
T-03	Output comparison — correct output	Program output matches expected.txt	Result: PASS	Unit
T-04	Output comparison — wrong output	Program output differs from expected.txt	Result: FAIL	Unit
T-05	Output comparison — whitespace/newline difference	Outputs differ only in trailing <code>\r\n</code> vs <code>\n</code>	Configurable; result consistent and documented	Unit
T-06	Missing source file inside ZIP	ZIP extracts but contains no main.c	Status: FILE NOT FOUND; processing continues	Unit

T-07	Corrupt or invalid ZIP file	ZIP file is malformed	Status: EXTRACTION ERROR; processing continues	Unit
T-08	Full pipeline — correct submission	Complete ZIP with valid C program	PASS result persisted to database	Integration
T-09	Database persistence after pipeline	Pipeline completed	StudentResult record exists in SQLite with correct fields	Integration
T-10	Config export and re-import round-trip	Config exported to file, re-imported on fresh state	Re-imported config produces identical pipeline behaviour	Integration
T-11	Batch of 5 mixed submissions — no crash	Directory with 5 varied ZIPs	5 results displayed; application stable throughout	System
T-12	Project reopen shows stored results	Project saved after batch run, app restarted	Identical results displayed without re-running pipeline	System
T-13	Installer creates desktop shortcut	Fresh Windows 10/11 machine	Shortcut present on desktop; app launches correctly	System
T-14	Help menu opens user manual	Application running	Manual opens and is readable	System
T-15	Full UAT scenario — R1 to R10	Clean install, prepared test data	All 10 requirements verified as Pass by independent evaluator	UAT

10. Future Improvements

The current version of the IAE satisfies all specified requirements. The following improvements are out of scope for this release but represent well-motivated directions for future development.

Multi-Test-Case Evaluation: The system currently supports a single expected output per project. A future version should allow multiple test cases per project, each with its own arguments and expected output, reporting a per-test-case breakdown per student. The database schema already anticipates this through the TEST_CASE and STUDENT_TEST_RESULT tables.

Resource Control: The pipeline enforces execution timeouts but places no limits on memory or CPU usage. A future version should add per-process resource limits and a dedicated `RESOURCE_LIMIT_EXCEEDED` status to prevent a misbehaving submission from degrading the host machine during a batch run.

Selective Re-Evaluation: Re-running a project to incorporate a late submission currently reprocesses every student. A future version should detect which submissions are new or changed and re-run only those, preserving existing results.

Audit Log: There is currently no record of when evaluations were run or whether settings changed after results were produced. An immutable audit trail per run would provide accountability in contexts where results carry grade consequences.

11. Conclusion

This design document has presented a comprehensive blueprint for the Integrated Assignment Environment (IAE), a standalone desktop application intended to automate the evaluation of programming assignments in an academic setting. Throughout the document, all ten requirements defined in the CE316 project specification have been addressed — from the creation and management of language configurations (Requirements 3–5) to the automated batch processing of student ZIP submissions (Requirement 6), the execution and output comparison pipeline (Requirements 7–8), persistent result reporting (Requirements 9–10), and the self-contained Windows installer with a built-in help manual (Requirements 1–2).

The system has been designed around the Model-View-Controller (MVC) architectural pattern, which cleanly separates the application's data layer, business logic, and user interface. This separation ensures that each module — the Configuration Manager, Submission Processor, Execution Engine, Output Comparator, and Report Manager — can be developed, tested, and maintained independently. SQLite has been chosen as the persistence layer to satisfy the no-server constraint, and JavaFX has been selected for the GUI to leverage the team's existing familiarity and the rich ecosystem of desktop UI components it provides.

The testing strategy, covering unit, integration, system, and user acceptance testing, provides a clear framework for verifying that the final implementation correctly and robustly fulfills all specified requirements.

The team is confident that this design provides a sufficient and practical foundation for implementing the IAE. The next step is to begin Phase 2 of the implementation plan, translating this design into a functional, tested, and deployable application.